

Capitolul 9

Valori proprii și valori singulare

9.1 Metode de tip iterație de putere

Pentru a găsi vectorul propriu dominant al matricei A , pornim cu un vector inițial x . Fiecare iterație constă din normalizarea vectorului curent și înmulțirea acestuia cu A . Câtul Rayleigh este folosit pentru a aproxima valoarea proprie. Comanda MATLAB `norm` simplifică implementarea, după cum se poate observa în codul de mai jos:

Algorithm (Iterația de putere).

Dându-se vectorul inițial x_0

for $j = 1, 2, 3, \dots$

$u_{j-1} = x_{j-1} / \|x_{j-1}\|_2$

$x_j = Au_{j-1}$

$\lambda_j = u_{j-1}^T Au_{j-1}$

end

$u_j = x_j / \|x_j\|_2$

```
function [lambda,u]=powerit(A,x,k)
for j=1:k
    u=x/norm(x);
    x=A*u;
    lambda=u'*x;
end
u=x/norm(x);
```

Pentru matricea $A = \begin{bmatrix} 1 & 3 \\ 2 & 2 \end{bmatrix}$, putem aplica iterația de putere astfel:

```
>> A=[1 3;2 2];
>> [lambda,u]=powerit(A,[-5;5],10)
```

Pentru a afla valoarea proprie a lui A cea mai apropiată de numărul real s , aplicăm iterația de putere pentru $(A - sI)^{-1}$ pentru a obține valoarea proprie cea mai mare în modul b a matricii $(A - sI)^{-1}$. Iterațiile de putere trebuie făcute folosind eliminarea gaussiană pentru $(A - sI)y_{k+1} = x_k$. Atunci $\lambda = b^{-1} + s$ este valoarea proprie a lui A cea mai apropiată de s . Vectorul propriu asociat valorii proprii λ este dat direct din calcul. Codul pentru iterația de putere inversă este:

Algoritm (Iterația de putere inversă).*Dându-se vectorul inițial x_0 și deplasarea s* **for** $j = 1, 2, 3, \dots$

$$u_{j-1} = x_{j-1} / \|x_{j-1}\|_2$$

$$\text{Rezolvăm } (A - sI)x_j = u_{j-1}$$

$$\lambda_j = u_{j-1}^T x_j$$

end

$$u_j = x_j / \|x_j\|_2$$

```
function [lambda,u]=invpowerit(A,x,s,k)
```

```
As=A-s*eye(size(A));
```

```
for j=1:k
```

```
    u=x/norm(x);
```

```
    x=As\u;
```

```
    lambda=u'*x;
```

```
end
```

```
lambda=1/lambda+s;
```

```
u=x/norm(x);
```

Putem aplica iterația de putere inversă pentru a afla valoarea proprie cea mai apropiată de -2 pentru matricea $A = \begin{bmatrix} 1 & 3 \\ 2 & 2 \end{bmatrix}$, astfel:

```
>> A=[1 3;2 2];
```

```
>> [lambda,u]=invpowerit(A,[-5;5],-2,10)
```

Iterația câtlui Rayleigh are următoarea implementare în MATLAB:

Algoritm (Iterația câtlui Rayleigh).*Dându-se vectorul inițial x_0* **for** $j = 1, 2, 3, \dots$

$$u_{j-1} = x_{j-1} / \|x_{j-1}\|_2$$

$$\lambda_{j-1} = u_{j-1}^T A u_{j-1}$$

$$\text{Rezolvăm } (A - \lambda_{j-1}I)x_j = u_{j-1}$$

end

$$u_j = x_j / \|x_j\|_2$$

```
function [lambda,u]=rqi(A,x,k)
```

```
for j=1:k
```

```
    u=x/norm(x);
```

```
    lambda=u'*A*u;
```

```
    x=(A-lambda*eye(size(A)))\u;
```

```
end
```

```
u=x/norm(x);
```

```
lambda=u'*A*u;
```

Pentru matricea $A = \begin{bmatrix} 1 & 3 \\ 2 & 2 \end{bmatrix}$, putem aplica iterația câtlui Rayleigh astfel:

```
>> A=[1 3;2 2];
```

```
>> [lambda,u]=rqi(A,[-5;5],1)
```

O iterație este suficientă, deoarece, la a doua iterație, valoarea lui `lambda` este foarte aproape de valoarea proprie, ceea ce face ca matricea $A - \lambda I$ să fie singulară.

9.2 Algoritmul QR

În cod MATLAB, algoritmul iterației simultane normalizate poate fi scris foarte ușor:

Algoritm (Iterația simultană normalizată).

```

Luăm  $\overline{Q_0} = I$ 
for  $j = 1, 2, 3, \dots$ 
     $A\overline{Q_j} = \overline{Q_{j+1}}R_{j+1}$ 
end

```

```

function lambda=nsi(A,k)
[m,n]=size(A);
Q=eye(m,m);
for j=1:k
    [Q,R]=qr(A*Q);
end
lambda=diag(Q'*A*Q);

```

Pentru a determina toate valorile proprii ale matricii $A = \begin{bmatrix} 1 & 3 \\ 2 & 2 \end{bmatrix}$, aplicăm iterația simultană normalizată astfel:

```

>> A=[1 3;2 2];
>> lambda=nsi(A,10)

```

Algoritmul QR nedeplasat calculează toate valorile proprii ale unei matrici. Codul MATLAB este dat de funcția de mai jos.

Algoritm (Algoritmul QR nedeplasat).

```

Luăm  $\overline{Q_0} = I, R_0 = A$ 
for  $j = 1, 2, 3, \dots$ 
     $R_j\overline{Q_j} = \overline{Q_{j+1}}R_{j+1}$ 
end

```

```

function lambda=unshiftedqr(A,k)
[m,n]=size(A);
Q=eye(m,m);
R=A;
for j=1:k
    [Q,R]=qr(R*Q);
end
lambda=diag(R*Q);

```

Pentru aceeași matrice, determinarea tuturor valorilor proprii se face astfel:

```

>> A=[1 3;2 2];
>> lambda=unshiftedqr(A,10)

```

Algoritmul QR deplasat este implementat de către codul MATLAB de mai jos. La fiecare pas, aplicăm pasul QR deplasat, și apoi verificăm rândul de jos. Dacă toate intrările sunt mici cu excepția intrării diagonale a_{nn} , declarăm acea intrare ca fiind o valoare proprie și efectuăm deflația prin ignorarea ultimului rând și a ultimei coloane în continuarea calculului nostru.

Algoritm (Algoritmul QR deplasat). *Fiecare pas constă din aplicarea deplasării, efectuarea unei factorizări QR, și apoi efectuarea unei deplasări înapoi. Mai precis,*

$$\begin{aligned}A_k - sI &= Q_k R_k \\ A_{k+1} &= R_k Q_k + sI.\end{aligned}$$

Care sunt alegerile bune pentru deplasarea s ? Această întrebare ne conduce la conceptul de deflație pentru calculul valorilor proprii. Vom alege deplasarea ca fiind intrarea din dreapta jos a matricii A_k . Aceasta va face ca iterația, pe măsură ce converge la forma Schur reală, să schimbe rândul de jos într-un rând de zerouri, cu excepția intrării din dreapta jos. După ce această intrare a devenit convergentă la o valoare proprie, vom efectua o deflație a matricii prin eliminarea ultimului rând și a ultimei coloane. Apoi continuăm cu găsirea celorlalte valori proprii.

```
function lambda=shiftedqr(A)
tol=1e-14;
m=size(A,1);
lambda=zeros(m,1);
n=m;
while n>1
    while max(abs(A(n,1:n-1)))>tol
        s=A(n,n);
        [Q,R]=qr(A-s*eye(n));
        A=R*Q+s*eye(n);
    end
    lambda(n)=A(n,n);
    n=n-1;
    A=A(1:n,1:n);
end
lambda(1)=A(1,1);
```

Pentru aceeași matrice, determinarea tuturor valorilor proprii se face astfel:

```
>> A=[1 3;2 2];
>> lambda=shiftedqr(A)
```

9.3 Descompunerea valorilor singulare

Comanda MATLAB pentru descompunerea valorilor singulare este `svd`, și

```
>> [U,S,V]=svd(A)
```

va returna toate cele trei matrici ale factorizării.

Probleme

1. Găsiți vectorul propriu dominant și valoarea proprie corespunzătoare pentru matricea A , folosind iterația de putere. Câte iterații sunt necesare pentru a obține patru zecimale exacte pentru valoarea proprie? Verificați rezultatul folosind funcția MATLAB `eig` sub forma

```
>> [V,D] = eig(A)
```

care returnează în V vectorii proprii ai matricii A și în D o matrice diagonală cu valorile proprii.

$$A = \begin{bmatrix} 10 & -12 & -6 \\ 5 & -5 & -4 \\ -1 & 0 & 3 \end{bmatrix}.$$

```
>> A=[10 -12 -6;5 -5 -4;-1 0 3];
```

2. Folosiți iterația de putere inversă pentru a găsi valoarea proprie din apropierea lui 1.95 și, respectiv, a lui 2.05, și vectorii proprii corespunzători, pentru matricea de la Problema 1.
3. Aplicați iterația cântului Rayleigh pentru matricea de la Problema 1. Încercați diferiți vectori inițiali până când găsiți toate cele trei valori proprii. Câte iterații sunt necesare pentru a obține patru zecimale exacte pentru valorile proprii?
4. Aplicați iterația simultană normalizată pentru matricea A de la Problema 1. Câte iterații sunt necesare pentru a obține patru zecimale exacte pentru valorile proprii?
5. Aplicați algoritmul QR nedeplasat pentru a găsi toate valorile proprii ale matricii:

$$A = \begin{bmatrix} 3 & 1 & -2 \\ 4 & 1 & 1 \\ -3 & 0 & 3 \end{bmatrix}.$$

Câte iterații sunt suficiente pentru a obține patru zecimale exacte?

```
>> A=[3 1 -2;4 1 1;-3 0 3]
```

6. Aplicați algoritmul QR deplasat cu toleranța 10^{-14} pentru a afla toate valorile proprii ale matricii de la Problema 5.
7. Folosiți comanda `svd` din MATLAB pentru a găsi cea mai bună aproximare de rang unu a matricii:

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 1 & 3 & 3 \\ 0 & 0 & 1 \end{bmatrix}.$$

Verificați rangul matricii obținute folosind comanda `rank` din MATLAB.

```
>> A=[1 2 4;1 3 3;0 0 1];
```

8. Găsiți cea mai bună aproximare de rang doi a matricii de la Problema 7. Verificați rangul matricii obținute.
9. Găsiți cea mai bună dreaptă în sensul celor mai mici pătrate care aproximează următorii vectori, și proiecțiile vectorilor pe spațiul unu-dimensional găsit.

$$\begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \\ 6 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 3 \end{bmatrix}.$$

10. Găsiți cel mai bun plan în sensul celor mai mici pătrate care aproximează vectorii de la Problema 9, și proiecțiile vectorilor pe spațiul bi-dimensional găsit.